

INTERNET TECHNOLOGY & WEB PAGE DESIGN

Unit-IV: CSS & JavaScript

Comprehensive Study Notes

HPU BCA Semester IV

1. Introduction to CSS

1.1 What is CSS?

CSS (Cascading Style Sheets) is a styling language used to describe the presentation of a document written in HTML or XML.

Definition: CSS is a style sheet language used for describing the presentation of a document written in a markup language.

1.2 CSS Versions

Version	Year	Key Features
CSS1	1996	Basic styling: fonts, colors, margins
CSS2	1998	Positioning, z-index, media types
CSS2.1	2004	Bug fixes, better browser support
CSS3	2011+	Modules: animations, flexbox, grid, transitions

2. Types of Style Sheets

2.1 Overview

Type	Location	Scope	Reusability
Inline	Within HTML tag	Single element	None
Internal	In <head> section	Single page	Single page
External	Separate .css file	Multiple pages	High

2.2 Inline Styles

Inline styles are applied directly to individual HTML elements using the `style` attribute.

```
<!DOCTYPE html>
<html>
<body>
  <h1 style="color:blue; text-align:center;">Hello World</h1>
  <p style="color:red; font-size:20px;">This is a red paragraph.</p>
  <div style="background-color:yellow;">This has a yellow background.</div>
</body>
</html>
```

2.3 Internal Styles

Internal styles are defined within the `<style>` tag in the `<head>` section.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    h1 { text-decoration: underline; text-align: center; }
    p.highlight { background-color: yellow; font-weight: bold; }
    .special { color: red; font-size: 24px; }
```

```
</style>
</head>
<body>
  <h1>Welcome to CSS</h1>
  <p class="highlight">This is a highlighted paragraph.</p>
  <p class="special">This is a special paragraph.</p>
</body>
</html>
```

2.4 External Styles

External styles are defined in a separate `.css` file and linked to the HTML document.

HTML File (index.html):

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This text is styled using an external CSS file.</p>
  <div class="box">This is a styled box.</div>
</body>
</html>
```

CSS File (style.css):

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
}
h1 {
  color: blue;
  border-bottom: 2px solid blue;
}
p {
  color: black;
  line-height: 1.6;
}
.box {
```

```
background-color: lightblue;  
border: 2px solid blue;  
border-radius: 10px;  
padding: 20px;  
}
```

3. CSS Selectors

3.1 Selector Types

Selector	Example	Description
Element	<code>p { }</code>	Selects all <code><p></code> elements
Class	<code>.classname { }</code>	Selects elements with class
ID	<code>#idname { }</code>	Selects element with ID
Universal	<code>* { }</code>	Selects all elements
Group	<code>h1, h2 { }</code>	Selects multiple elements
Descendant	<code>div p { }</code>	Selects <code><p></code> inside <code><div></code>
Child	<code>div > p { }</code>	Selects direct child <code><p></code>
Attribute	<code>a[target] { }</code>	Selects elements with attribute

3.2 Pseudo-classes

Pseudo-classes define a special state of an element.

Pseudo-class	Example	Description
<code>:hover</code>	<code>a:hover { }</code>	Element on mouse over
<code>:focus</code>	<code>input:focus { }</code>	Element with focus
<code>:first-child</code>	<code>p:first-child { }</code>	First child element
<code>:nth-child(n)</code>	<code>tr:nth-child(even) { }</code>	Nth child element

```
/* Pseudo-class examples */  
a:hover {
```

```
    color: red;
    text-decoration: underline;
}
input:focus {
    border: 2px solid blue;
    outline: none;
}
tr:nth-child(even) {
    background-color: #f2f2f2;
}
```

3.3 Pseudo-elements

Pseudo-elements style specific parts of an element.

Pseudo-element	Example	Description
::before	p::before { }	Insert content before element
::after	p::after { }	Insert content after element
::first-line	p::first-line { }	First line of text
::first-letter	p::first-letter { }	First letter of text

```
/* Pseudo-element examples */
p::before {
    content: ">> ";
    color: blue;
}
p::after {
    content: " <<";
    color: blue;
}
p::first-letter {
    font-size: 200%;
    color: red;
}
```

4. CSS Properties

4.1 Text Properties

Property	Values	Description
color	name, hex, rgb	Text color
font-size	px, em, rem, %	Font size
font-family	Arial, serif, etc.	Font family
font-weight	bold, normal, 100-900	Font weight
text-align	left, center, right	Text alignment
text-decoration	underline, none	Text decoration
line-height	1.5, 2, etc.	Line height

4.2 Background Properties

Property	Description
background-color	Sets background color
background-image	Sets background image
background-repeat	Repeat mode (repeat, no-repeat)
background-position	Position of background
background-size	Size of background (cover, contain)

4.3 Box Properties

Property	Description
width	Element width
height	Element height
margin	Space outside element
padding	Space inside element
border	Border around element
border-radius	Rounded corners

4.4 Layout Properties

Property	Values	Description
display	block, inline, flex, grid, none	Display behavior
position	static, relative, absolute, fixed, sticky	Positioning method
float	left, right, none	Float element
clear	left, right, both	Clear float

4.5 Positioning Examples

```
/* Static (default) */
.static { position: static; }

/* Relative - positioned relative to its normal position */
.relative { position: relative; top: 20px; left: 20px; }

/* Absolute - positioned relative to nearest positioned ancestor */
.absolute { position: absolute; top: 0; right: 0; }

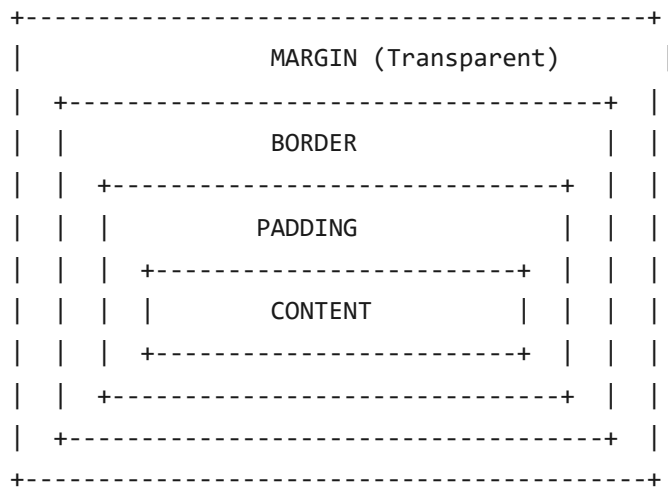
/* Fixed - fixed relative to viewport */
.fixed { position: fixed; bottom: 0; right: 0; }
```

```
/* Sticky - sticks on scroll */  
.sticky { position: sticky; top: 0; }
```

5. CSS Box Model

5.1 What is the Box Model?

The box model is the fundamental layout concept in CSS. Every HTML element is treated as a rectangular box with four layers:



Layer	Description	Properties
Content	The actual content (text, images)	width , height
Padding	Space between content and border	padding , padding-top , etc.
Border	Border around padding	border , border-width , border-color
Margin	Space outside border	margin , margin-top , etc.

5.2 Box Model Example

```
.box {  
  width: 300px;  
  height: 200px;
```

```
padding: 20px;  
border: 5px solid #2e86c1;  
margin: 30px auto;  
background-color: #f0f3f4;  
}
```

```
<div class="box">  
  This box demonstrates the CSS box model.  
  Width = 300px + 40px (padding) + 10px (border) = 350px total  
</div>
```

6. Advantages and Limitations of CSS

6.1 Advantages

Advantage	Description
Separation of Concerns	Separates content from presentation
Consistency	Consistent styling across pages
Reusability	One style sheet for multiple pages
Maintainability	Easy to update and maintain
Page Load Speed	Faster loading, smaller HTML files
Accessibility	Better for screen readers and devices
Flexibility	Multiple layout options
Device Optimization	Responsive design for different devices

6.2 Limitations

Limitation	Description
Browser Compatibility	Different browsers render CSS differently
Security	CSS can be manipulated by users
Complexity	Complex layouts require advanced knowledge
Performance	Heavy CSS can slow down rendering
Vertical Control	Limited vertical alignment options

7. Flexbox and Grid Layout

7.1 Flexbox Layout

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  gap: 20px;  
  flex-wrap: wrap;  
}  
.item {  
  flex: 1;  
  min-width: 200px;  
  padding: 20px;  
  background: #f0f3f4;  
  border: 1px solid #2e86c1;  
}
```

```
<div class="container">  
  <div class="item">Box 1</div>  
  <div class="item">Box 2</div>  
  <div class="item">Box 3</div>  
</div>
```

7.2 Grid Layout

```
.grid-container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
  padding: 20px;  
}  
.grid-item {  
  background: #f0f3f4;  
  border: 1px solid #2e86c1;  
  padding: 20px;  
  text-align: center;
```

```
}  
@media (max-width: 768px) {  
  .grid-container {  
    grid-template-columns: 1fr;  
  }  
}
```

8. Introduction to JavaScript

8.1 What is JavaScript?

JavaScript is a lightweight, interpreted, object-oriented programming language used to create interactive and dynamic content on web pages.

Definition: JavaScript is a client-side scripting language that enables dynamic behavior on web pages when a user interacts with them.

8.2 Features of JavaScript

- **Lightweight:** Minimal overhead, runs in browser
- **Interpreted:** No compilation required
- **Event-driven:** Responds to user actions
- **Object-oriented:** Supports objects and prototypes
- **Cross-platform:** Works on all major browsers
- **Dynamic typing:** Variable types are determined at runtime
- **DOM manipulation:** Can modify HTML and CSS dynamically

8.3 Data Types

Data Type	Description	Example
Number	Integer or floating point	42 , 3.14
String	Sequence of characters	"Hello" , 'World'
Boolean	True or false	true , false
Null	Intentional absence of value	null
Undefined	Variable not assigned a value	undefined

Object	Collection of key-value pairs	{name: "John"}
Array	Ordered list of values	[1, 2, 3]

8.4 Variables

```

<script>
  // var (function-scoped, older way)
  var name = "John";

  // let (block-scoped, preferred for mutable variables)
  let age = 25;
  let city = "Shimla";

  // const (block-scoped, cannot be reassigned)
  const PI = 3.14159;
  const college = "HPU";

  // Dynamic typing - same variable can hold different types
  let data = 42;           // number
  data = "Hello";         // now a string
  data = true;            // now a boolean
</script>

```

8.5 Operators

Category	Operators	Example
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code>	<code>x + y</code>
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>	<code>x += 5</code>
Comparison	<code>==</code> , <code>===</code> , <code>!=</code> , <code>!==</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	<code>x === y</code>
Logical	<code>&&</code> , <code> </code> , <code>!</code>	<code>x > 0 && y > 0</code>
String	<code>+</code> (concatenation)	<code>"Hello " + name</code>

Ternary	? :	age >= 18 ? "Adult" : "Minor"
----------------	-----	----------------------------------

```
<script>
  let a = 10, b = 3;
  console.log(a + b); // 13
  console.log(a - b); // 7
  console.log(a * b); // 30
  console.log(a / b); // 3.33
  console.log(a % b); // 1
  console.log(a ** b); // 1000

  // Strict vs loose equality
  console.log(5 == "5"); // true (loose)
  console.log(5 === "5"); // false (strict)
</script>
```

9. Control Structures

9.1 Conditional Statements

```
<script>
  // if-else
  let marks = 85;
  if (marks >= 90) {
    console.log("Grade: A");
  } else if (marks >= 75) {
    console.log("Grade: B");
  } else {
    console.log("Grade: C");
  }

  // switch
  let day = 3;
  switch(day) {
    case 1: console.log("Monday"); break;
    case 2: console.log("Tuesday"); break;
    case 3: console.log("Wednesday"); break;
    default: console.log("Other day");
  }
</script>
```

9.2 Loops

```
<script>
  // for loop
  for (let i = 0; i < 5; i++) {
    console.log("Iteration: " + i);
  }

  // while loop
  let count = 0;
  while (count < 3) {
    console.log("Count: " + count);
    count++;
  }
</script>
```

```
// do-while loop
let x = 0;
do {
    console.log("x = " + x);
    x++;
} while (x < 3);

// for...of (arrays)
let fruits = ["Apple", "Banana", "Cherry"];
for (let fruit of fruits) {
    console.log(fruit);
}
</script>
```

10. Functions in JavaScript

10.1 Function Declaration

```
<script>
  // Function declaration
  function greet(name) {
    return "Hello, " + name + "!";
  }

  // Function expression
  const add = function(a, b) {
    return a + b;
  };

  // Arrow function (ES6)
  const multiply = (a, b) => a * b;

  // Calling functions
  console.log(greet("Rahul"));      // Hello, Rahul!
  console.log(add(10, 20));         // 30
  console.log(multiply(5, 4));      // 20
</script>
```

10.2 Parameters and Return

```
<script>
  // Function with default parameters
  function welcome(name = "Guest") {
    return "Welcome " + name;
  }

  console.log(welcome("Priya"));    // Welcome Priya
  console.log(welcome());           // Welcome Guest

  // Function with multiple return values
  function calculator(a, b) {
    return {
      sum: a + b,
```

```
        diff: a - b,  
        prod: a * b  
    };  
}  
  
let result = calculator(10, 5);  
console.log(result.sum);    // 15  
console.log(result.diff);   // 5  
</script>
```

10.3 Events

Event	Description	Example
onclick	Element is clicked	button onclick="myFunc()"
onmouseover	Mouse moves over element	div onmouseover="handler()"
onmouseout	Mouse leaves element	div onmouseout="handler()"
onkeydown	Key is pressed	input onkeydown="handler()"
onload	Page finishes loading	body onload="init()"
onsubmit	Form is submitted	form onsubmit="validate()"
onchange	Value of element changes	select onchange="handler()"

```
<!DOCTYPE html>  
<html>  
<head>  
  <script>  
    function displayMessage() {  
      alert("Button was clicked!");  
    }  
    function showText(text) {  
      document.getElementById("output").innerHTML = text;  
    }  
  </script>  
</head>  
<body>  
  <button onclick="displayMessage()">Click Me</button>  
  <button onmouseover="showText('Mouse Over')">Hover Me</button>
```

```
    <p id="output"></p>  
</body>  
</html>
```

11. DOM Manipulation

11.1 What is DOM?

The **Document Object Model (DOM)** is a programming interface for HTML documents. It represents the page as a tree of objects that can be manipulated with JavaScript.

11.2 Accessing Elements

Method	Description	Example
<code>getElementById()</code>	Select by ID	<code>document.getElementById("header")</code>
<code>getElementsByClassName()</code>	Select by class	<code>document.getElementsByClassName("item")</code>
<code>getElementsByTagName()</code>	Select by tag	<code>document.getElementsByTagName("p")</code>
<code>querySelector()</code>	Select first match (CSS selector)	<code>document.querySelector(".class")</code>
<code>querySelectorAll()</code>	Select all matches	<code>document.querySelectorAll("p")</code>

11.3 innerHTML and Style

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function changeContent() {
      // Change content using innerHTML
      document.getElementById("message").innerHTML =
```



```
        "<strong>Content updated!</strong>";

        // Change style
        document.getElementById("message").style.color = "green";
        document.getElementById("message").style.fontSize = "24px";
        document.getElementById("message").style.backgroundColor = "#f0f3f4";
    }

    function resetContent() {
        document.getElementById("message").innerHTML = "Original content";
        document.getElementById("message").style.color = "black";
        document.getElementById("message").style.backgroundColor = "transparent";
    }
</script>
</head>
<body>
    <p id="message">Original content</p>
    <button onclick="changeContent()">Change</button>
    <button onclick="resetContent()">Reset</button>
</body>
</html>
```

11.4 Event Listeners

```
<!DOCTYPE html>
<html>
<head>
    <script>
        // Modern way: addEventListener
        function init() {
            let btn = document.getElementById("myBtn");
            btn.addEventListener("click", function() {
                alert("Button clicked via event listener!");
            });

            let box = document.getElementById("myBox");
            box.addEventListener("mouseover", function() {
                this.style.backgroundColor = "#2e86c1";
                this.style.color = "white";
            });
            box.addEventListener("mouseout", function() {
                this.style.backgroundColor = "#f0f3f4";
                this.style.color = "black";
            });
        }
    </script>
</head>
<body>
    <button id="myBtn">Click Me</button>
    <div id="myBox">
        <p>Mouseover and Mouseout Event Listeners</p>
    </div>
</body>
</html>
```

```
        });  
    }  
  
    // Execute after page loads  
    window.onload = init;  
</script>  
</head>  
<body>  
    <button id="myBtn">Click with Listener</button>  
    <div id="myBox" style="padding:20px;border:1px solid #333;">  
        Hover over me!  
    </div>  
</body>  
</html>
```

12. Form Validation using JavaScript

12.1 Client-side Form Validation

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function validateForm() {
      let name = document.getElementById("name").value;
      let email = document.getElementById("email").value;
      let password = document.getElementById("password").value;
      let age = document.getElementById("age").value;
      let error = "";

      // Name validation
      if (name == "" || name.length < 3) {
        error += "Name must be at least 3 characters.\n";
      }

      // Email validation
      let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
      if (!emailPattern.test(email)) {
        error += "Please enter a valid email address.\n";
      }

      // Password validation
      if (password.length < 6) {
        error += "Password must be at least 6 characters.\n";
      }

      // Age validation
      if (age == "" || isNaN(age) || age < 1 || age > 120) {
        error += "Please enter a valid age (1-120).\n";
      }

      if (error != "") {
        alert(error);
        return false;
      }
      alert("Form submitted successfully!");
    }
  </script>
</head>
</html>
```

```
        return true;
    }
</script>
</head>
<body>
    <h3>Registration Form with Validation</h3>
    <form onsubmit="return validateForm()">
        Name: <input type="text" id="name"><br>
        Email: <input type="email" id="email"><br>
        Password: <input type="password" id="password"><br>
        Age: <input type="number" id="age"><br>
        <input type="submit" value="Register">
    </form>
</body>
</html>
```

12.2 Real-time Validation

```
<script>
    function validateEmail() {
        let email = document.getElementById("email").value;
        let error = document.getElementById("emailError");
        let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

        if (pattern.test(email)) {
            error.innerHTML = "Valid email";
            error.style.color = "green";
        } else {
            error.innerHTML = "Invalid email format";
            error.style.color = "red";
        }
    }

    function checkPassword() {
        let pwd = document.getElementById("password").value;
        let msg = document.getElementById("pwdError");

        if (pwd.length < 6) {
            msg.innerHTML = "Too short (" + pwd.length + "/6)";
            msg.style.color = "red";
        } else {
            msg.innerHTML = "Strong enough";
            msg.style.color = "green";
        }
    }
```

```
}  
</script>
```

13. Questions

Short Answer Questions (2-5 Marks)

1. What is CSS?
2. What are the types of style sheets?
3. What is the difference between inline and internal CSS?
4. What is the CSS box model?
5. What are CSS selectors?
6. What is JavaScript?
7. What are the data types in JavaScript?
8. What is the difference between `let` , `const` , and `var` ?
9. What is DOM?
10. What is the difference between `==` and `===` ?

Long Answer Questions (10 Marks)

1. Explain CSS and its advantages and limitations.
2. Explain the CSS box model with examples.
3. Explain different types of style sheets with examples.
4. Explain CSS positioning and layout properties.
5. Explain JavaScript functions with examples (declaration, parameters, return, events).
6. Explain DOM manipulation with `getElementById`, `innerHTML`, and style properties.
7. Explain form validation using JavaScript with examples.
8. Explain CSS selectors with examples of each type.

Program Writing Questions (5 Marks)

Q1. Write a CSS file to style a web page with a header, navigation, main content, and footer.

```
* { margin: 0; padding: 0; box-sizing: border-box; }  
body { font-family: Arial, sans-serif; line-height: 1.6; }
```

```
header { background: #2e86c1; color: white; padding: 20px; }
nav { background: #1a5276; padding: 10px; }
nav a { color: white; text-decoration: none; padding: 10px; }
nav a:hover { background: #2e86c1; }
main { padding: 20px; min-height: 400px; }
footer { background: #1a5276; color: white; text-align: center; padding: 10px; }
```

Q2. Write CSS to create a responsive webpage layout using flexbox.

```
.container { display: flex; flex-wrap: wrap; gap: 20px; }
.box { flex: 1; min-width: 250px; padding: 20px;
      background: #f0f3f4; border: 1px solid #2e86c1; }
@media (max-width: 768px) {
  .container { flex-direction: column; }
}
```

Q3. Write a JavaScript function to validate a registration form.

```
function validateForm() {
  let name = document.getElementById("name").value;
  let email = document.getElementById("email").value;
  if (name == "" || email == "") {
    alert("Name and email are required!");
    return false;
  }
  return true;
}
```

Objective Questions (1 Mark)

1. CSS stands for:
(a) Cascading Style Sheets (b) Computer Style Sheets
(c) Creative Style Sheets (d) None
2. The CSS box model includes:
(a) Margin, Border, Padding, Content (b) Margin, Border, Padding only
(c) Border, Padding, Content only (d) None
3. Which CSS property is used for text color?
(a) color (b) text-color (c) font-color (d) background-color

4. Which JavaScript keyword declares a block-scoped variable?
- (a) var (b) let (c) Both (d) None
5. The method to select an element by ID is:
- (a) getElementById() (b) getElementByClass()
(c) getElementByTag() (d) None

Quick Revision

CSS Summary

Category	Key Properties
Text	color , font-size , font-family , text-align
Background	background-color , background-image
Box	width , height , margin , padding , border
Layout	display , position , flex , grid , float
Effects	opacity , box-shadow , transition

JavaScript Quick Reference

```
// Variables
let x = 10;
const PI = 3.14;

// Function
function add(a, b) { return a + b; }

// Arrow function
const square = n => n * n;

// DOM access
document.getElementById("id");
document.querySelector(".class");

// Event listener
element.addEventListener("click", handler);

// Form validation
if (value == "") { alert("Required!"); return false; }
```

Box Model Quick Reference

```
Total width = width + padding-left + padding-right  
              + border-left + border-right  
Total height = height + padding-top + padding-bottom  
              + border-top + border-bottom
```

End of Unit-IV Notes